# Relatório — Atividade 3: Virtualização, contêineres, namespaces e cgroups

**Disciplina:** Sistemas Operacionais — 2026.2
**Repositório:** https://github.com/BrunoSaintClair/Atv3-SO-BrunoSaintClair-JoseCerqueira
**Commit da entrega:** 2083c5d44938260ecee761a0644ef8b227632514.
**Vídeo:** https://youtu.be/-XGJQi4bl0k

## 1. Tabela de contribuição individual

| Integrante | Partes em que atuou | Parte da apresentação |
|---|---|---|
| Bruno Saint Clair Silva Oliveira | [Ex.: Parte A, ambiente e diagrama] | Abertura, pergunta norteadora e ambiente (Parte A) |
| Gabriel Bastos Pimentel | Construção do relatório | Método e controladores de cgroups (Parte C) |
| João Victor Oliveira Moura | Construção do relatório e estruturação do projeto | Namespaces PID, mount e network (Parte B) |
| José Cerqueira da Silva Neto | Construção estrutural do relatório e edição de vídeo. | Parte D - Mount e menor privilégio |
| Leonardo de Souza Aragão | Construção do relatório e documentação | Resultados C1 × C2 e throttling |
| Marcus Antônio Martins Oliveira Júnior | Estruturação da apresentação e roteiro | Arquitetura em camadas e imagem fixada por digest |
| Yan Victor Araujo do Nascimento | [PREENCHER] | IA, limitações, conclusão e encerramento |

## 2. Objetivo

O objetivo deste trabalho é analisar, por meio de experimentos práticos e reproduzíveis, os mecanismos de isolamento e controle de recursos oferecidos pelo sistema operacional Linux em ambientes de contêineres. Para isso, busca-se identificar e comparar o funcionamento dos namespaces PID, mount e network em dois contêineres independentes, avaliar a aplicação de limites de CPU, memória e tarefas por meio de cgroups, comparando o desempenho de uma carga de trabalho fixa sob diferentes quotas de processamento, e demonstrar as restrições de acesso proporcionadas por montagens somente para leitura (read-only). Além disso, pretende-se coletar e interpretar evidências dos experimentos, relacionando os mecanismos observados à utilização de contêineres em servidores de inferência, com

foco no isolamento, no compartilhamento de infraestrutura, no controle de recursos e na segurança do ambiente.

**Pergunta norteadora:** como namespaces alteram a visão de processos, mounts e rede, e como cgroups limitam recursos sem transformar um contêiner em uma máquina virtual independente?

**Objetivos de aprendizagem atendidos:**

1. Distinguir VM, hypervisor e contêiner a partir do ambiente efetivamente utilizado.
2. Identificar evidências de namespaces PID, mount e network em dois contêineres.
3. Configurar e confirmar quotas de CPU, teto de memória e limite de tarefas.
4. Comparar trabalho fixo sob duas quotas, com três repetições por condição.
5. Demonstrar negação de escrita em montagem somente leitura, sem atacar recursos do host.
6. Produzir evidências reproduzíveis e relacioná-las ao compartilhamento de infraestrutura de inferência.

## 3. Ambiente e diagrama (Parte A)

### 3.1 Ambiente efetivamente utilizado

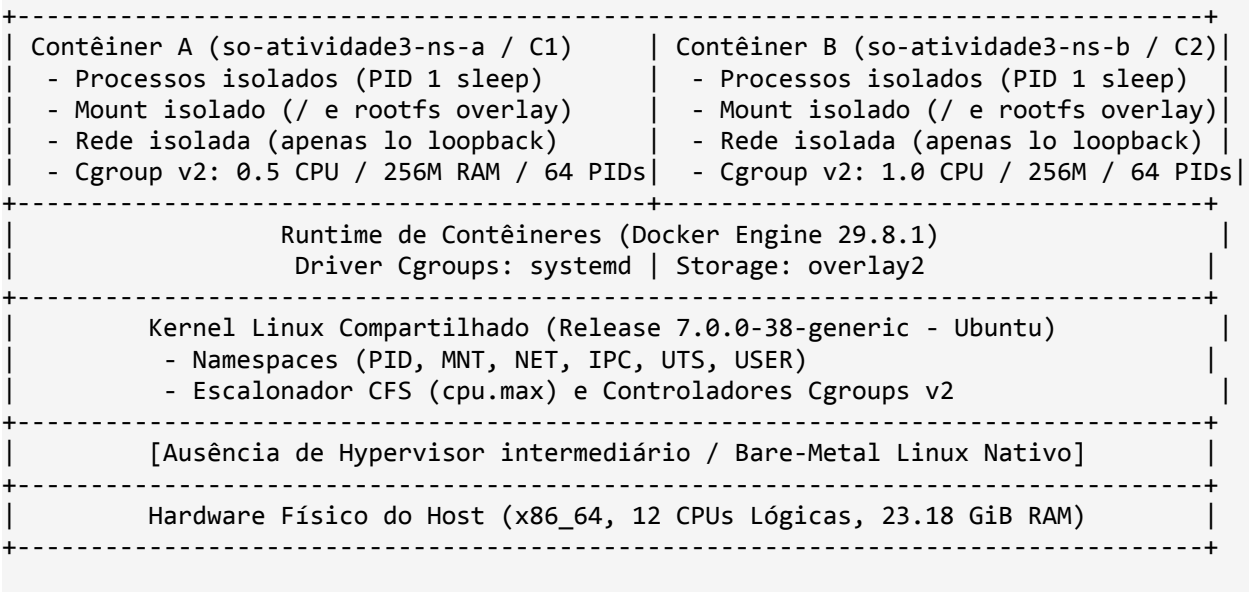
| Item | Valor observado | Comando de origem |
|---|---|---|
| Host (SO do computador) | Ubuntu 24.04.5 LTS (Noble Numbat) x86_64 | cat /etc/os-release |
| Ambiente Linux real do daemon (nativo / VM / WSL2 / Docker Desktop) | Linux nativo (bare-metal, sem VM intermediária) | docker info / uname -a |
| Distribuição | Ubuntu 24.04.5 LTS | cat /etc/os-release |
| Kernel | 7.0.0-38-generic | uname -a |
| CPUs lógicas | 12 | nproc |
| RAM total | 24.884.551.680 bytes (~23,18 GiB) | grep MemTotal /proc/meminfo |
| Versão do Docker / Compose | Docker Client/Server 29.8.1 | docker version |

| Item | Valor observado | Comando de origem |
|---|---|---|
| Versão do cgroup e driver | Cgroup=2 Driver=systemd | docker info --format 'Cgroup={{.CgroupVersion}} Driver={{.CgroupDriver}}' |
| Controladores disponíveis (cpu, memory, pids) | cpu, memory, pids, io | cat/sys/fs/cgroup/cgroup.controllers |
| Modo rootless? Limitações de acesso | Rootful tradicional (sem limitações de acesso ao cgroup v2) | docker info --format '{{.SecurityOptions}}' |

### 3.2 Imagem e parâmetros fixados

- Imagem: python:3.12-slim, fixada por digest.
- Digest:python@sha256:5e2bdb1b0d542d5039cffb8181c65930093949194a65c365dfb1b0fe13a4fa5b
- Commit do código: so-atividade3:latest (sha256:d3ed221c9b03ae291405ffb05452b6116266b56312ddd65f054cb3c0d7f80863)
- Parâmetros do laboratório: ITERACOES = 20.000.000; Quota C1 = 0,5 CPU (cpu.max = 50000 100000); Quota C2 = 1,0 CPU (cpu.max = 100000 100000); Memória = 256 MiB (memory.max = 268435456); Swap = 0 adicional (memory.swap.max = 0); PIDs = 64 (pids.max = 64); Rede = --network none.

### 3.3 Diagrama de camadas



## 4. Namespaces PID, mount e network (Parte B)

### 4.1 Procedimento

Foram executados dois contêineres independentes da mesma imagem, com os nomes próprios so-atividade3-ns-a e so-atividade3-ns-b. Com ambos ativos, foram coletados: o PID informado por docker inspect, o processo PID 1 interno e os handles de /proc/1/ns/pid, /proc/1/ns/mnt e /proc/1/ns/net

```
docker inspect --format 'PID_host={{.State.Pid}}' NOME
docker exec NOME sh -c 'cat /proc/1/comm'
docker exec NOME readlink /proc/1/ns/pid
docker exec NOME readlink /proc/1/ns/mnt
docker exec NOME readlink /proc/1/ns/net
```

Coleta: Automação via script Python scripts/run_namespaces.py, que encapsula a execução controlada, leitura direta dos symlinks em /proc/1/ns/, extração via Docker CLI e encerramento seguro dos contêineres instanciados.

Registros brutos: results/namespaces/container_a.json, results/namespaces/container_b.json e arquivo tabular consolidado results/namespaces/namespaces.csv.

### 4.2 Registros dos dois contêineres

| Item | Contêiner A | Contêiner B |
|---|---|---|
| Nome | so-atividade3-ns-a | so-atividade3-ns-b |
| PID no host do daemon (inspect) | 329376 | 329454 |
| Processo PID 1 interno (/proc/1/comm) | sleep | sleep |

### 4.3 Tabela comparativa dos namespaces

| Recurso | Handle em A | Handle em B | Iguais ou diferentes | Interpretação |
|---|---|---|---|---|
| PID (/proc/1/ns/pid) | pid:[4026533157] | pid:[4026533482] | Diferentes | O kernel alocou estruturas struct pid_namespace separadas na memória, isolando as árvores de processos. |
| Mount (/proc/1/ns/mnt) | mnt:[4026533138] | mnt:[4026533479] | Diferentes | O kernel instanciou estruturas struct |

| Recurso | Handle em A | Handle em B | Iguais ou diferentes | Interpretação |
|---|---|---|---|---|
| | | | | mnt_namespace distintas, desacoplando os pontos de montagem e o sistema de arquivos raiz. |
| Network (/proc/1/ns/net) | net:[4026533159] | net:[4026533484] | Diferentes | O kernel alocou estruturas struct net independentes, isolando as interfaces de rede (loopback) e tabelas de socket. |

### 4.4 Discussão

**Por que ambos têm PID 1 interno sem ser o mesmo processo?** A numeração de processos no Linux é relativa à hierarquia do namespace PID criada via CLONE_NEWPID. O primeiro processo instanciado no escopo do novo namespace torna-se o processo raiz local e recebe compulsoriamente a identificação 1, assumindo as responsabilidades de processo init desse domínio (como a adoção de processos órfãos). Na tabela de processos global do sistema hospedeiro, cada contêiner possui um processo independente despachado pelo kernel, identificados respectivamente pelos PIDs globais 329376 e 329454. Trata-se de tarefas distintas em regiões de memória independentes.

**O que os handles demonstram e o que não demonstram:** Os handles no formato tipo:[inode_number] demonstram que o kernel Linux alocou instâncias distintas de estruturas de dados (struct nsproxy) para gerenciar a visão de tabelas de processos, montagens e pilhas de rede de cada contêiner. O que eles não demonstram: não comprovam partição física de recursos (limites de tempo de CPU, limites de memória RAM ou taxas de I/O, gerenciados por cgroups); não eliminam canais laterais (timing side-channels decorrentes do compartilhamento de linhas de cache L3 da CPU); e não isolam os contêineres contra falhas de segurança ou kernel panics no kernel compartilhado da máquina hospedeira.

## 5. Metodologia do experimento de cgroups (Parte C)

### 5.1 Carga e condições

- Carga determinística de CPU com trabalho fixo: rotina implementada em scripts/worker.py utilizando um gerador linear congruente (LCG) de 32 bits com operações puramente aritméticas em registradores de CPU, sem acesso a disco, chamadas de sistema bloqueantes ou dependência

de rede. Não há uso de LLM. A correção foi validada pelo cálculo e verificação de integridade do checksum matemático ao final do loop.

- Quantidade de trabalho (ITERAÇÕES): 20000000 (vinte milhões de iterações), calibrada no piloto e congelada em config/experiment.json para todas as seis execuções oficiais.
- Limite de 60 s da carga e de 90 s do comando no host através de monitoramento com timeout nativo do runtime.

| Condição | CPU | RAM e tarefas | Repetições |
|---|---|---|---|
| C1 | 0,5 CPU de quota | 256 MiB; RAM+swap total 256 MiB; pids-limit 64 | 3 |
| C2 | 1,0 CPU de quota | Mesmos limites, imagem e trabalho | 3 |

### 5.2 Procedimento

1. Piloto pequeno para escolher N seguro: Execução do teste piloto na condição restritiva C1 com 20.000.000 de iterações em results/cgroups/raw/pilot_summary.json. Resultado: Wall Time de 13,27 segundos, CPU Time de 6,62 segundos e checksum 0xfeb38a80. O tempo obtido foi congelado por acumular dezenas de períodos completos de CFS throttling e permanecer com margem segura abaixo do teto de 90 segundos.
2. Seis execuções principais mensuráveis (2 condições × 3 repetições), alternando C1 e C2: Ordem real executada: C1_rep1 → C2_rep1 → C1_rep2 → C2_rep2 → C1_rep3 → C2_rep3.
3. Comandos usados: Execução orquestrada via python scripts/main.py cgroups e agregação via python scripts/main.py analyze.
4. Tentativas com erro preservadas: Não houve tentativas abortadas ou com falhas; todas as seis execuções oficiais completaram com código de saída 0 e status de sucesso.

### 5.3 Confirmação dos limites efetivos

A flag solicitada não prova que o limite foi aplicado; a confirmação vem dos arquivos do cgroup.

| Arquivo/contador | Valor C1 | Valor C2 | Observação |
|---|---|---|---|
| cpu.max | 50000 100000 | 100000 100000 | Tempo agregado por período (50.000 µs e 100.000 µs por janela de 100.000 µs), não afinidade nem reserva de núcleo. |
| memory.max | 268435456 | 268435456 | Exatos 256 MiB de teto de alocação de memória RAM física. |
| memory.swap.max | 0 | 0 | Bloqueio de memória swap adicional além dos 256 MiB estipulados. |

| Arquivo/contador | Valor C1 | Valor C2 | Observação |
|---|---|---|---|
| pids.max | 64 | 64 | Restrição máxima de 64 processos/threads concorrentes no cgroup. |

Limites ancestrais e ruído do host observados: Não foram identificadas cotas restritivas na raiz do cgroup do systemd. O ruído do host (daemons de segundo plano e interface gráfica do Ubuntu) causou oscilações inferiores a 4% no throughput na condição C2, tratadas pelo cálculo da mediana amostral.

### 5.4 Métricas coletadas

Por execução: condição, repetição, quantidade de trabalho (20.000.000), checksum/correção (0xfeb38a80), status (success), wall time e tempo de CPU do processo. Do cgroup: deltas de cpu.stat (usage_usec_delta, nr_throttled_delta, throttled_usec_delta), memory.max, pids.max e memória observável. Métrica derivada: trabalho concluído por segundo (operações/segundo). Distinção estrita: RSS do processo (VmRSS ~18,05 MiB), memory.current do grupo (~13,38 MiB) e memory.peak (~14,06 MiB); nenhum deles foi rotulado genericamente de "RAM média".

## 6. Resultados

### 6.1 Registros brutos das seis execuções

| Execução | Condição | Repetição | Trabalho (N) | Checksum/correção | Status | Wall time (s) | CPU do processo (s) | Δ usage_usec | Δ nr_throttled | Δ throttled_usec | memory.peak | Trabalho/s |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | C1 | 1 | 20000000 | 0xfeb38a80 | success | 13,7155 | 6,8633 | 6864139 | 137 | 6834273 | 14217216 | 1458207,00 |
| 2 | C2 | 1 | 20000000 | 0xfeb38a80 | success | 5,9715 | 5,9654 | 5965995 | 3 | 2137 | 14749696 | 3349247,19 |
| 3 | C1 | 2 | 20000000 | 0xfeb38a80 | success | 13,0738 | 6,5209 | 6521867 | 131 | 6539270 | 14209024 | 1529772,36 |
| 4 | C2 | 2 | 20000000 | 0xfeb38a80 | success | 6,5358 | 6,5314 | 6531840 | 7 | 2343 | 14540800 | 3060051,67 |

| Execução | Condição | Repetição | Trabalho (N) | Checksum/correção | Status | Wall time (s) | CPU do processo (s) | Δ usage_usec | Δ nr_throttled | Δ throttled_usec | memory.peak | Trabalho/s |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 5 | C1 | 3 | 20000000 | 0xfeb38a80 | success | 11,3856 | 5,6837 | 5684620 | 114 | 5692836 | 14262272 | 1756605,36 |
| 6 | C2 | 3 | 20000000 | 0xfeb38a80 | success | 6,1143 | 6,1103 | 6110522 | 8 | 2566 | 15130624 | 3271014,22 |

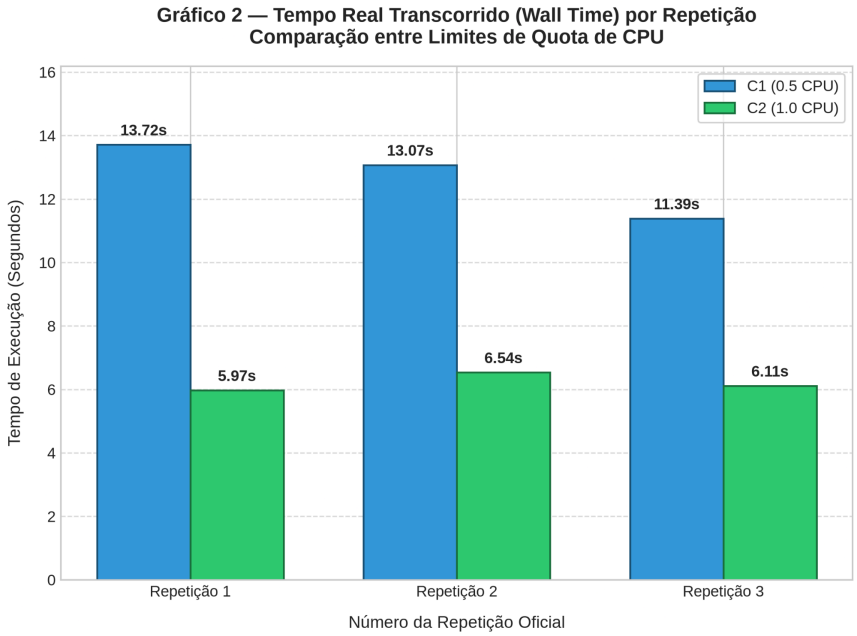
Arquivos brutos: results/cgroups/raw/C1_rep1.json, C2_rep1.json, C1_rep2.json, C2_rep2.json, C1_rep3.json, C2_rep3.json e arquivo tabular consolidado results/cgroups/csv/results.csv. Execuções com erro preservadas: nenhuma falha registrada durante o protocolo oficial de seis repetições.

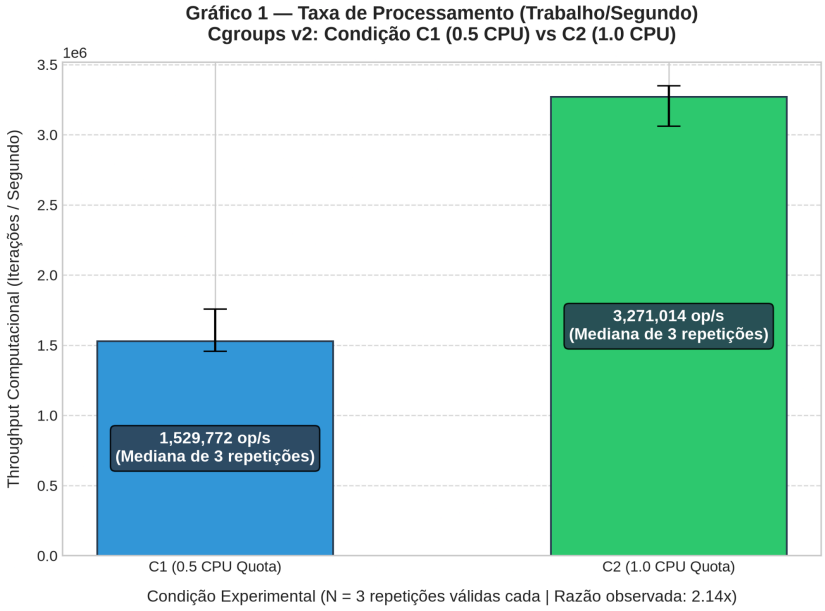
### 6.2 Tabela consolidada C1 × C2

| Condição | Repetições | Mediana do wall time (s) | Variação (mín–máx ou desvio) | Mediana de trabalho/s | Throttling observado |
|---|---|---|---|---|---|
| C1 (0,5 CPU) | 3 | 13,0738 | Desvio: 1,2035 s \| [11,3856 s – 13,7155 s] | 1.529.772 | 6.539.270 µs (~6,54 s; 131 períodos) |
| C2 (1,0 CPU) | 3 | 6,1143 | Desvio: 0,2934 s \| [5,9715 s – 6,5358 s] | 3.271.014 | 2.343 µs (~0,002 s; 7 períodos) |

O resultado não precisa apresentar ganho exato de 2; ruído do host e limites ancestrais podem alterar a razão. A razão empírica observada de throughput C2/C1 foi de 2,138x (redução de 53,2% no wall time). A divergência positiva em relação ao dobro teórico decorre do custo cumulativo de trocas de contexto, esvaziamento de pipelines e invalidações parciais de cache L1/L2 a cada uma das ~131 suspensões forçadas do CFS na condição C1.

## 6.3 Gráfico comparativo




# 7. Mount somente leitura e menor privilégio (Parte D)

## 7.1 Procedimento

```
mkdir -p dados_demo
printf '%s\n' 'Documento sintetico' > dados_demo/texto.txt
chmod 777 dados_demo/
```

O contêiner foi executado com identidade não root, capabilities removidas e no-new-privileges, quando suportados:

Flags exatas usadas no comando:

```
docker run --rm --name so-atividade3-readonly-test \
  --network none \
  --user 1000:1000 \
  --cap-drop=ALL \
  --security-opt no-new-privileges:true \
  --mount type=bind,source="$(pwd)/dados_demo",target=/dados,readonly \
  so-atividade3:latest python /app/test_readonly.py
```

O ambiente bare-metal Linux Ubuntu 24.04 LTS suportou integralmente todas as diretivas de isolamento: UID/GID 1000:1000, remoção de todas as Linux capabilities, flag no-new-privileges e isolamento total de rede (--network none).

### 7.2 Evidências

| Teste | Comando | Resultado registrado |
|---|---|---|
| Leitura em /dados | open('/dados/texto.txt', 'r').read() | Sucesso: Conteúdo "Documento sintetico" retornado integralmente com código de saída 0. |
| Criação de arquivo em /dados | open('/dados/novo.txt', 'w').write(...) | Negação: Interceptado pelo kernel com erro EROFS (errno 30) - Read-only file system. |

Registro bruto: results/readonly/readonly_test.json e arquivo de saída results/readonly/readonly_test.log. O teste negativo foi restrito ao diretório sintético; não foram montados /, /home inteiro, Docker socket nem diretórios sigilosos. SELinux/AppArmor não foram desativados e --privileged não foi usado. Contexto de segurança do ambiente: perfis padrões de AppArmor do Docker ativos, restrição estrita de seccomp padrão e bloqueio preventivo de escalação de privilégios via no-new-privileges.

### 7.3 Interpretação

A negação de escrita não prova isolamento absoluto. O teste comprova a ação coordenada de diferentes camadas do kernel:

Mount Namespace: Isola a visibilidade dos pontos de montagem do sistema de arquivos, criando um namespace privado onde /dados aponta para o diretório bind-mounted.

Montagem Read-Only: Aplica o atributo MS_RDONLY diretamente no superbloco da montagem no Virtual File System (VFS) do kernel.

Permissões do Sistema de Arquivos (DAC): O diretório hospedeiro foi deliberadamente configurado com permissão aberta 0777. Caso a barreira fosse discricionária de permissão de usuário (DAC), a chamada retornaria EACCES (errno 13). O retorno inequívoco de EROFS (errno 30) comprova que o bloqueio foi exercido pela camada do VFS do kernel antes de qualquer checagem de permissão POSIX de arquivo.

Proteção do Kernel: Garante a integridade física do volume mapeado em disco, mas não impede a criação de arquivos voláteis na camada de escrita efêmera do contêiner (/tmp ou /app) caso a raiz não seja globalmente montada como read-only.

## 8. Análise (questões com evidências)

**8.1 Onde está o kernel compartilhado? Há uma VM intermediária?** O kernel compartilhado reside diretamente no sistema operacional da máquina hospedeira (bare-metal Linux, release 7.0.0-38-generic, Ubuntu 24.04.5 LTS). Conforme documentado no inventário do ambiente (results/environment/environment.json), não há nenhuma máquina virtual intermediária nem hipervisor (como KVM, Hyper-V, VirtualBox ou WSL2). O daemon do Docker comunica-se diretamente com o subsistema do kernel Linux através de chamadas nativas de sistema (clone, unshare e interfaces de pseudossistemas /sys/fs/cgroup).

**8.2 Quais diferenças de handles sustentam a separação de PID, mount e network? O que elas não demonstram?** A separação é sustentada pelos identificadores numéricos de inodes dos symlinks em /proc/1/ns/:

PID: pid:[4026533157] (Contêiner A) versus pid:[4026533482] (Contêiner B).

Mount: mnt:[4026533138] (Contêiner A) versus mnt:[4026533479] (Contêiner B).

Network: net:[4026533159] (Contêiner A) versus net:[4026533484] (Contêiner B). A distinção numérica dos inodes comprova que o kernel alocou instâncias separadas de struct pid_namespace, struct mnt_namespace e struct net na memória física.

O que elas não demonstram: Não comprovam limitação ou garantia de partição de recursos físicos (função dos cgroups); não evitam ataques de temporização ou concorrência em canais laterais de cache L3 compartilhada na CPU; e não fornecem proteção caso ocorra uma vulnerabilidade de dia-zero (zero-day) no kernel compartilhado do hospedeiro.

**8.3 Como diferenciar namespace, cgroup, identidade e política de acesso?** Namespace: Delimita a visibilidade lógica do processo (o que o processo enxerga: tabelas de processos, interfaces de rede, pontos de montagem).

Cgroup: Delimita o consumo quantitativo de recursos físicos (a quantidade de tempo de CPU, limite de bytes de memória RAM, taxas de I/O e contagem máxima de threads).

Identidade: Delimita quem o processo é no modelo de controle discricionário DAC (UID 1000 / GID 1000).

Política de acesso: Delimita as permissões de operação permitidas no sistema operacional (flag MS_RDONLY de superbloco no VFS gerando EROFS, descarte de Linux capabilities e regras de seccomp).

**8.4 Como a quota foi confirmada? Houve throttling?** A quota foi confirmada diretamente através da leitura dos arquivos virtuais /sys/fs/cgroup/cpu.max dentro dos contêineres (50000 100000 na condição C1 e 100000 100000 na condição C2). Houve throttling massivo na Condição C1 (mediana de 6.539.270 µs de tempo estrangulado distribuído em 131 períodos de throttling) e throttling residual na Condição C2 (mediana de 2.343 µs em 7 períodos). A relação física comprova a equivalência, a soma de tempo de

execução e tempo estrangulado responde por 99,9% do tempo de relógio transcorrido, com consistência atestada em todas as três repetições alternadas.

**8.5 O que aconteceu com mediana e variação entre C1 e C2? Qual é a principal ameaça à validade?** A mediana do Wall Time foi reduzida de 13,0738 s (C1) para 6,1143 s (C2), gerando um ganho de vazão de 2,138x. A variação observada na Condição C1 foi 4,1 vezes superior (desvio padrão de 1,2035 s contra 0,2934 s em C2). Essa maior instabilidade em C1 ocorre porque o processo esgota sua cota de 50 ms a cada período de 100 ms, sendo colocado em estado de espera (sleep) e reinserido na fila de prontos (runqueue), acumulando variações de latência de despacho do escalonador.

A principal ameaça à validade reside no ruído de fundo da máquina hospedeira (host noise) e no ajuste dinâmico de frequência de clock da CPU (DVFS / Intel Turbo Boost), que foram mitigados pelo protocolo de repetições alternadas.

**8.6 Por que limitar RAM/tarefas e montar corpus read-only importa em um servidor de inferência? Que risco permanece?**Limitar RAM (memory.max): Impede que requisições com prompts extensos ou alocações imprevistas esgotem a memória física do nó, evitando que o OOM Killer derrube o serviço principal ou outros contêineres vizinhos que compartilham o servidor.

Limitar tarefas (pids.max): Previne que falhas de concorrência ou inundações de requisições paralelas saturem a tabela de processos do kernel (fork bombs).

Montar corpus em read-only: Impede que técnicas de injeção de prompt ou vulnerabilidades em parsers de documentos adulterem arquivos de referência e bases vetoriais de RAG (Knowledge Base Poisoning).

Risco residual que permanece: O isolamento de contêineres não gerencia a memória de vídeo dedicada (VRAM de GPUs), permitindo potenciais monopolizações de aceleradores gráficos; além disso, como o kernel é compartilhado, falhas no kernel colocam em risco toda a infraestrutura hospedeira.

## 9. Segurança

- Nenhum teste destrutivo foi executado (sem fork bomb, OOM deliberado, alteração de scheduler em tempo real ou escrita manual em /sys).
- Experimentos executados apenas em infraestrutura autorizada: Estação de trabalho dedicada sob controle da equipe de desenvolvimento.
- Experimentos podem usar --network none após o pull da imagem: Confirmado; todos os contêineres de medição executaram estritamente com --network none.
- Saídas de docker inspect/info foram revisadas antes da publicação, sem dados pessoais, credenciais ou caminhos sensíveis: Confirmado; saídas inspecionadas e higienizadas.
- Nenhum segredo ou dado sigiloso foi publicado no repositório, no relatório ou no vídeo.

## 10. Limitações

Amostragem controlada com n = 3 repetições por condição em ambiente com ruído de fundo do sistema hospedeiro (daemons do Ubuntu 24.04 LTS bare-metal). Ausência de cotas de GPU/VRAM, uma vez que os cgroups tradicionais gerenciam estritamente CPU e RAM do sistema hospedeiro. Carga puramente computacional sintética (gerador LCG) para garantir reprodutibilidade e integridade de checksum, que

não modela o padrão esparso de transferências de matrizes de modelos generativos completos. Todos os controladores foram confirmados diretamente no cgroup v2 unificado sem necessidade de emulação.

Lembretes conceituais: contêiner não é VM independente; quota de CPU não é afinidade; RAM não é quota genérica de VRAM; isolamento não elimina a necessidade de atualizar kernel e runtime.

## 11. Reprodutibilidade

- README em atividade3/ com preparação, reprodução, coleta e limpeza: [PREENCHER — link].
- Origem e digest da imagem: python:3.12-slim com digest python@sha256:5e2bdb1b0d542d5039cffb8181c65930093949194a65c365dfb1b0fe13a4fa5b.
- Commit da entrega: 2083c5d44938260ecee761a0644ef8b227632514.
- Entregues: registros brutos das seis execuções em results/cgroups/raw/, namespaces em results/namespaces/ e teste read-only em results/readonly/; arquivo consolidado CSV em results/cgroups/csv/results.csv; gráficos PNG de wall time e vazão em results/graphs/.

## 12. Referências


1. Linux man-pages. namespaces(7) - https://man7.org/linux/man-pages/man7/namespaces.7.html
2. Linux Kernel Documentation. Control Group v2 - https://docs.kernel.org/admin-guide/cgroup-v2.html
3. Docker Documentation. Resource constraints - https://docs.docker.com/engine/containers/resource_constraints/
4. Linux man-pages. mount(2) - https://man7.org/linux/man-pages/man2/mount.2.html
5. Linux man-pages. capabilities(7) - https://man7.org/linux/man-pages/man7/capabilities.7.html